

封装

类级别的封装：寻找属性、行为、过程

package级别的封装：

1. 按照业务、功能模块进行封装；

以人力管理系统为例

基础信息模块（员工、组织、职位） --- com.xxxx.base

薪资模块 --- com.xxxx.salary

时间模块 --- com.xxxx.time

员工关系模块 --- com.xxxx.relationship

..... --

针对模块，我们希望能做到：高内聚，低耦合

2. 按照代码职责进行划分

MVC模式：Model View Control，即--模型层，视图层，控制层

View 视图层一般不会使用java编写（俺曾经使用郭swing...及其难用

Control 控制层：控制数据流转与界面切换 数据来了要发给谁处理？处理完了有哪些数据要展示，要在那个页面上显示...

Model 模型层，又分为两层

1. Service 业务层： 处理业务功能和事务控制

2. DAO/Mapper 持久层：与数据库进行数据交互

按照这种方式进行封装，结构大约如下：

com.xxxx.view view会调用controller

com.xxxx.controller controller会调用service

com.xxxx.service service又会调用mapper

com.xxx.mapper

所以，这种封装方式实际上是会耦合在一起的

3. 混合方式

最常用的方式是混合使用上述两种模式

业务、功能还是划分的

每个功能的内部按照MVC进行划分

比如：

com.xxxx.base

base.controller

```
base.service
base.mapper
com.xxxx.salary
salary.controller
salary.service
salary.mapper
.....
```

访问权限修饰符

private 私有

无关键字 默认(英文中或称为package/friendly)

protected 保护

public 公有

	类的内部	同一个包	不同包的子类	任意类
private	○	X	X	X
默认	○	○	X	X
protected	○	○	○	X
public	○	○	○	○

java中的包也是一个权限控制结构啊...如果不是private，只要两个类在一个包里，即使没有继承关系也能互相调用属性、方法

Class的权限，只能是public或默认

属性和方法则四种权限都能用

static静态关键字

静态的特点：

1. 存储在元空间里，自始至终只有唯一的一份，不随实例的数量变化而变化
2. 静态内容可以通过类名直接访问
3. 静态方法只能访问静态资源

.class被加载到元空间的触发方式：

1. new了这个类或者其子类的实例
2. 调用了这个类的静态方法或者静态属性
3. 通过反射机制Class.forName("")手动加载.class

静态代码块

```
static{  
    // code  
}  
  
// 可以用于对复杂静态变量的初始化  
// 也会在class被加载的时候执行
```

final 最终关键字

可以用于修饰：

1. 类 ： final类不允许被继承
2. 局部变量：常量，第一次被赋值后不许修改，通常使用全大写字母写局部变量
3. 方法 ：这个方法不允许再被重写了（继承了这个类，不允许再override该final方法）

单例模式

// 饿汉单例

```
public class HungrySingleton
{

    private static final instance = new HungrySingleton();

    private HungrySingleton(){}

    public static HungrySingleton getInstance()
    {
        return instance;
    }
}
```

// 懒汉单例

```
public class LazySingleton
{
    private static LazySingleton instance;
    private LazySingleton(){}

    public static LazySingleton getInstance()
    {
        if(instance == null)
        {
            synchronized(LazySingleton.class)
            {
                if(instance == null)
                {
                    instance = new LazySingleton();
                }
            }
        }
        return instance;
    }
}
```

